



Parul University

**FACULTY OF ENGINEERING AND
TECHNOLOGY
BACHELOR OF TECHNOLOGY**

Pattern Recognition
Laboratory
(303105482)

7TH SEMESTER

Artificial Intelligence and Data Science Department

Laboratory Manual

CERTIFICATE

This is to certify that

M Venkata Vara Prasad with Enrollment no 2303031240082 has successfully completed his laboratory experiments in the subject (subject code) of Pattern recognition laboratory(303105482) from the department of CSE in Artificial intelligence and Data science during the academic year 2026- 2027



Date of Submission:

Staff In Charge:

Head of Department:

TABLE OF CONTENT

Sr. No	Experiment Title	Page No		Date of Start	Date of Completion	Sign	Marks
		From	To				
1.	Implementation of Gradient Descent						
2.	Implementation of Linear Regression using Gradient Descent						
3.	Comparison of Classification Accuracy of SVM for given dataset						
4.	Generate your own feature set by combining existing set of features, or defining new ones. Feature Representation						
5.	Generate samples of a normal distribution with specific parameters with respect to Mean and Covariance						
6.	Implement Linear Perceptron Learning algorithm						
7.	Build IRIS flower classification in python using pattern recognition models						

Practical no:1

Aim:1. Implementation of Gradient Descent.

What is Gradient Descent?

Gradient Descent stands as a cornerstone orchestrating the intricate dance of model optimization. At its core, it is a numerical optimization algorithm that aims to find the optimal parameters—weights and biases—of a neural network by minimizing a defined cost function.

Gradient Descent (GD) is a widely used optimization algorithm in machine learning and deep learning that minimises the cost function of a neural network model during training. It works by iteratively adjusting the weights or parameters of the model in the direction of the negative gradient of the cost function until the minimum of the cost function is reached.

The learning happens during the backpropagation while training the neural network-based model. There is a term known as Gradient Descent, which is used to optimize the weight and biases based on the cost function. The cost function evaluates the difference between the actual and predicted outputs.

Gradient Descent is a fundamental optimization algorithm in machine learning used to minimize the cost or loss function during model training.

- It iteratively adjusts model parameters by moving in the direction of the steepest decrease in the cost function.
- The algorithm calculates gradients, representing the partial derivatives of the cost function concerning each parameter.

These gradients guide the updates, ensuring convergence towards the optimal parameter values that yield the lowest possible cost.

Gradient Descent is versatile and applicable to various machine learning models, including linear regression and neural networks. Its efficiency lies in navigating the parameter space efficiently, enabling models to learn patterns and make accurate predictions. Adjusting the learning rate is crucial to balance convergence speed and avoiding overshooting the optimal solution.

Gradient descent is an iterative optimization algorithm for finding the local minimum of a function.

To find the local minimum of a function using gradient descent, we must take steps proportional to the negative of the gradient (move away from the gradient) of the function at the current point. If we take steps proportional to the positive of the gradient (moving towards the gradient), we will approach a local maximum of the

function, and the procedure is called **Gradient Ascent**.

Explanation

Function and Derivative Definition:

The $f(x)$ function represents the cost or loss function to be minimized. $f(x)=x^2-4x+6$

The $df(x)$ function calculates its derivative, which is the gradient.

Gradient Descent Function:

- It initializes x to `initial_x`.
- It iteratively updates x for a specified number of `num_iterations`.
- In each iteration, it calculates the gradient using $df(x)$.
- It then updates x by subtracting the product of the `learning_rate` and the gradient. The `learning_rate` controls the step size in the direction of the negative gradient.

Parameter Setting and Execution:

- `initial_x_value`: The starting point for the optimization.
- `learning_rate_value`: Determines how large a step is taken in each iteration. A smaller learning rate can lead to slower convergence but might prevent overshooting the minimum. A larger learning rate can lead to faster convergence but risks instability or divergence.
- `iterations`: The number of times the update rule is applied.

Visualization:

The code plots the function and the path taken by the x value during the gradient descent process, illustrating its convergence towards the minimum.

Program:

```
import numpy as np
import matplotlib.pyplot as plt

# 1. Define the function to minimize (e.g., a simple quadratic function)
def f(x):
    return x**2 - 4*x + 6

# 2. Define the derivative (gradient) of the function
def df(x):
    return 2*x - 4

# 3. Implement the gradient descent algorithm
```

```
def gradient_descent(initial_x, learning_rate, num_iterations):  
    x = initial_x  
    x_history = [x] # To store the values of x at each iteration  
  
    for i in range(num_iterations):  
        gradient = df(x) # Calculate the gradient  
        x = x - learning_rate * gradient # Update x  
        x_history.append(x)  
  
    return x, x_history  
  
# 4. Set parameters and run the algorithm  
initial_x = 0  
learning_rate = 0.1  
num_iterations = 50  
  
final_x, x_history = gradient_descent(initial_x, learning_rate, num_iterations)  
  
print(f"Local minimum found at x: {final_x:.2f}")  
  
# 5. Visualize the process (optional)  
x_vals = np.linspace(-1, 5, 100)  
plt.plot(x_vals, f(x_vals), label='f(x)')  
plt.plot(x_history, [f(val) for val in x_history], 'rx-', label='Gradient Descent Path')  
plt.xlabel('x')  
plt.ylabel('f(x)')  
plt.title('Name :M Venkata Vara Prasad ,  
ERP:2303031240082,  
ROLL NO: 61 ')  
plt.legend()  
plt.grid(True)  
plt.show()
```

Output:

Local minimum found at x: 2.00

